SUBJECT: Deep Learning (JNTUH)
UNIT: V
TOPICS: Practical Methodology & Applications
PART 1: PRACTICAL METHODOLOGY

1. PERFORMANCE METRICS

Definition:
- Metrics are the standard units of measurement used to evaluate how well a model is performing.
- Using the right metric is crucial because optimization algorithms (like SGD) will try to minimize error based on this metric.

Common Metrics:

A. Classification Metrics:
- Accuracy: Percentage of correct predictions. Good for balanced datasets (e.g., 50% cat, 50% dog).
- Precision & Recall: Used for unbalanced datasets (e.g., Cancer detection).
  * Precision: Of all positives predicted, how many were actually positive?
  * Recall: Of all actual positives, how many did we find?
- F1-Score: The harmonic mean of Precision and Recall.

B. Regression Metrics:
- Mean Squared Error (MSE): Average squared difference between predicted and actual values. Common for price prediction.

C. Specialized Metrics:
- BLEU Score: Used in Machine Translation (NLP) to compare machine output vs human translation.

Exam Tip: Always mention that "Accuracy is not enough for skewed datasets."

---

2. DEFAULT BASELINE MODELS

Definition:
- A baseline is a simple, standard model built first to establish a reference performance. You should not start with complex architectures.

Recommended Baselines:
- For Computer Vision: A simple Convolutional Neural Network (CNN) with 2-3 layers, ReLU activation, and Batch Normalization.
- For Sequence Data (Text/Audio): A standard LSTM or GRU with one hidden layer.
- Optimizer: Adam (adaptive learning rate) or SGD with Momentum.
- Regularization: Early Stopping and Dropout (to prevent overfitting).

This document is available on studocu

Downloaded by J Naresh Kumar (naresh5021@gmail.com)

3. DETERMINING WHETHER TO GATHER MORE DATA

Process:

After building a baseline, look at the Training vs. Test error.

Case A: High Training Error (Underfitting)

- Problem: The model is too simple to learn the data.
- Solution: DO NOT get more data yet. Increase model complexity (more layers, more neurons) or tune hyperparameters.

Case B: Low Training Error but High Test Error (Overfitting)

- Problem: The model memorized the training data but fails on new data.
- Solution: Gather more data. This is the only time "more data" is the primary fix.
- Alternatives to more data: Data Augmentation, Regularization (Dropout), or Transfer Learning.

---

4. SELECTING HYPERPARAMETERS

Definition:

- Hyperparameters are settings chosen BEFORE training (e.g., Learning Rate), unlike weights which are learned DURING training.

Strategies for Selection:

A. Manual Tuning:
  - Based on intuition and experience. Hard to reproduce.

B. Grid Search:
  - Try every combination of parameters (e.g., LR=[0.1, 0.01], Batch=[32, 64]).
  - Very slow and computationally expensive.

C. Random Search:
  - Randomly sample combinations.
  - Often faster and more effective than Grid Search because some parameters matter more than others.

Key Hyperparameters:

- Learning Rate (Most important): Controls step size in Gradient Descent.
- Batch Size: Affects speed and noise of gradients.
- Number of Epochs: How long to train.

---

5. EXAMPLE: MULTI-DIGIT NUMBER RECOGNITION

Problem Statement:

- Recognize a sequence of numbers from an image (e.g., Street view house numbers).

Approach:

1. Baseline: Train a CNN to classify single digits. Use a sliding window to

Downloaded by J Naresh Kumar (naresh5021@gmail.com)

detect digits one by one.
2. Metric: Sequence Accuracy (The whole number must be correct, not just parts).
3. Advanced Model: Use a CRNN (Convolutional Recurrent Neural Network).
- CNN layers extract visual features.
- RNN layers interpret the sequence and order of digits.

---

6. DEBUGGING STRATEGIES

Why is it hard?
- In DL, code can run without crashing but produce wrong results (logic errors).

Strategies:
A. Visualize:
- Look at the images the model fails on.
- Visualize feature maps to see what the CNN "sees".

B. Overfit a Small Batch:
- Train on just 10-20 examples.
- The model should reach 100% accuracy quickly. If not, there is a bug in the code/logic.

C. Monitor Gradients:
- Check if gradients are becoming NaN (Exploding) or Zero (Vanishing).
- Use Gradient Clipping for exploding gradients.

PART 2: APPLICATIONS

1. LARGE-SCALE DEEP LEARNING

Concept:
- Deep Learning scales well: More Data + Bigger Models = Better Performance.

Hardware Approaches:
A. GPUs (Graphics Processing Units):
- Essential for DL. Optimized for parallel Matrix Multiplications.

B. Distributed Training:
- Data Parallelism: Split the dataset across multiple GPUs. Each GPU calculates gradients for its chunk, then they average them.
- Model Parallelism: Split a giant model across multiple GPUs (e.g., Layer 1 on GPU 1, Layer 2 on GPU 2).

Model Compression:
- Techniques to run big models on phones (Mobile/Edge).
- Knowledge Distillation: Train a small "Student" model to mimic the output of a large "Teacher" model.

---

2. COMPUTER VISION (CV)

This document is available on studocu

Downloaded by J Naresh Kumar (naresh5021@gmail.com)

Definition:
- Teaching computers to interpret images and video.

Core Tasks:
- Classification: What is in the image? (Cat vs Dog)
- Object Detection: Where is it? (Bounding Box)
- Segmentation: Which pixels belong to the object? (Pixel-level mask)

Key Techniques:
- Preprocessing: Normalizing pixel values (0-255 -> 0-1) and Contrast Normalization (GCN) to handle lighting differences.
- Augmentation: Flipping, rotating, and cropping images to create more training data and reduce overfitting.

---

3. SPEECH RECOGNITION (ASR)

Definition:
- Automatic Speech Recognition: Converting Audio Waveforms -> Text.

Evolution:
- Old Way (GMM-HMM): Used statistical probabilities to map sounds (phonemes) to words.
- Deep Learning Way: Uses Deep Neural Networks (DNNs) and Recurrent Neural Networks (LSTMs).
- End-to-End Models: Systems like DeepSpeech map raw audio directly to text using CTC (Connectionist Temporal Classification) loss, removing manual alignment steps.

---

4. NATURAL LANGUAGE PROCESSING (NLP)

Definition:
- Processing human language (Text).

Key Concepts:
- N-Grams: Predicting the next word based on the previous N-1 words.
- Neural Language Models (NLMs): Using Neural Networks to predict probabilities.
- Word Embeddings: Converting words into dense vectors (numbers).
  * Words with similar meanings have vectors that are close in space.
  * Example: Vector("King") - Vector("Man") + Vector("Woman") approx Vector("Queen").
- Hierarchical Softmax: An optimization technique to speed up prediction when the vocabulary is huge (millions of words). Instead of calculating probability for every word, it uses a tree structure.

Downloaded by J Naresh Kumar (naresh5021@gmail.com)

Deep Learning Unit V Important Questions

Practical methodology and performance

1. Explain the difference between the training loss and the real-world performance metric of a deployed system. Give examples of metrics for classification and detection tasks.
2. What are default baseline models? Why is it important to build a simple end-to-end baseline before developing complex deep models?
3. Discuss how you decide whether to gather more data versus improving the model architecture or regularization. Mention signs of data problems vs. model capacity problems.

Hyperparameters, debugging, multi-digit example

4. List key hyperparameters in deep learning (e.g., learning rate, batch size, depth, width, regularization coefficients) and describe a systematic procedure for tuning them.
5. Describe a step-by-step debugging strategy for a deep learning project: from sanity checks on labels and loss, to overfitting a small subset, to monitoring gradients and activations.
6. In the context of multi-digit number recognition (e.g., house numbers), outline the full pipeline: data preprocessing, model architecture, loss function, evaluation metric, and error analysis.

Large-scale deep learning and applications

7. What are the main computational challenges in large-scale deep learning, and how are they addressed using distributed training, GPUs/TPUs, and data pipelines?
8. Describe at least three major applications of deep learning in computer vision (e.g., classification, detection, segmentation) and the typical models used.
9. Explain how deep learning is applied in speech recognition and speech generation. What kinds of architectures and features are commonly used?
10. Discuss deep learning applications in natural language processing such as machine translation, question answering, and language modeling, mentioning suitable architectures (RNNs, Transformers, Seq2Seq).
11. Give examples of “other” real-world applications of deep learning (e.g., recommender systems, healthcare, robotics, finance) and briefly explain what makes deep learning suitable for each.

This document is available on studocu

Downloaded by J Naresh Kumar (naresh5021@gmail.com)

1. Explain the difference between the training loss and the real-world performance metric of a deployed system. Give examples of metrics for classification and detection tasks.

In deep learning, *training loss* refers to the optimization objective minimized during model training—typically a differentiable scalar function such as cross-entropy or mean squared error used to adjust weights via gradient-based learning. However, this loss is not always identical to the *real-world performance metric* that determines how well the system behaves when deployed in actual environments. Real-world performance uses domain-specific, task-appropriate measures—often non-differentiable or discontinuous—that directly reflect user requirements, such as accuracy for classification, Intersection-over-Union (IoU) for segmentation, mean Average Precision (mAP) for detection, or BLEU score for machine translation. A model may achieve a very low training loss but still perform poorly when confronted with class imbalance, distribution shift, noisy data, or evaluation metrics that emphasize different error patterns. Thus, practitioners must distinguish between the convenient surrogate loss optimized during training and the true operational metric assessed during inference.

---

2. What are default baseline models? Why is it important to build a simple end-to-end baseline before developing complex deep models?

A *default baseline model* is a simple, easily implementable system used as the initial reference point for any deep learning project. It could be a logistic regression classifier, a single-layer CNN, a bag-of-words model, or even a constant-value predictor—anything that can be constructed quickly and provides a basic performance level. Building such a baseline is crucial because it establishes a lower bound against which complex models can be meaningfully compared. Without a baseline, one risks developing large, over-engineered architectures that perform no better than trivial alternatives. Baselines also reveal data issues such as label noise, class imbalance, or flawed preprocessing pipelines. Moreover, they help verify that the task itself is learnable and ensure that evaluation metrics are implemented correctly. Starting simple allows systematic progress, clearer debugging, and a more principled exploration of architectural improvements.

---

3. Discuss how you decide whether to gather more data versus improving the model architecture or regularization. Mention signs of data problems vs. model capacity problems.

The decision between collecting more data and improving model architecture depends primarily on diagnosing the pattern of training and validation errors. If a model exhibits very low training error but high validation error, this suggests overfitting and indicates that adding regularization, reducing model capacity, or gathering more diverse data may help. Conversely, if both training and validation errors are high, the model is underfitting, meaning it lacks sufficient capacity or appropriate inductive biases, and architectural improvements may be necessary. Signs of data problems include inconsistent labels, unstable training curves, sudden drops in accuracy, and models that perform worse as they grow in size. Model-capacity problems appear when deeper or wider networks consistently reduce training loss. Understanding this trade-off helps allocate resources effectively: improving data quality when noise dominates, or enhancing the architecture when the model lacks expressiveness.

Downloaded by J Naresh Kumar (naresh5021@gmail.com)

4. List key hyperparameters in deep learning and describe a systematic procedure for tuning them.

Key hyperparameters include the learning rate, batch size, depth and width of the network, regularization coefficients (L1/L2 weight decay), dropout rate, optimizer-specific parameters (momentum, betas in Adam), learning-rate schedule, number of epochs, and data-augmentation settings. A systematic tuning procedure begins with establishing a baseline model and selecting a coarse grid or random search range for the most sensitive parameters, especially learning rate and batch size. After identifying viable regions, finer search or Bayesian optimization can refine the optimal values. Practitioners typically adjust one "high-impact" hyperparameter at a time while keeping others fixed to preserve experimental clarity. Monitoring validation curves helps prevent under- or over-fitting, and early stopping ensures efficient resource utilization. Careful logging and reproducibility practices complete the systematic tuning workflow.

---

5. Describe a step-by-step debugging strategy for a deep learning project.

Debugging a deep learning pipeline proceeds through structured stages. First, verify dataset integrity by checking for corrupted images, incorrect labels, or inconsistent dimensions. Next, implement sanity checks such as ensuring the model can overfit a tiny subset of data—typically 20–100 samples—confirming gradient flow, loss implementation, and data preprocessing. Then inspect activations and gradients to detect saturation, exploding values, or dead neurons. Visualize batches, intermediate maps, and sample predictions to verify semantic correctness. If training fails to progress, compare with a simpler baseline or reduce model complexity. Gradually reintroduce components such as data augmentation or advanced regularizers. Finally, track training dynamics using tools like TensorBoard and intervene using gradient clipping, alternative optimizers, or learning-rate warmups if necessary. This systematic approach isolates issues quickly and avoids guess-based troubleshooting.

---

6. In multi-digit number recognition (e.g., house numbers), outline the full pipeline: preprocessing → architecture → loss → evaluation → error analysis.

A multi-digit recognition pipeline begins with preprocessing: normalizing pixel intensities, resizing input images, cropping around digits, applying data augmentation, and, when necessary, labeling variable-length sequences of digits. A typical model architecture uses a CNN feature extractor followed by a recurrent layer (e.g., LSTM/GRU) or CTC-based decoder to handle sequences, or alternatively a transformer encoder for attention-based alignment. The loss function depends on the decoding method: CTC loss for unsegmented sequences or cross-entropy for explicit digit classification. Evaluation metrics include sequence accuracy, per-digit accuracy, and edit distance. Error analysis involves categorizing failures into occlusions, overlapping digits, illumination variation, segmentation mistakes, and generalization issues on rare digit combinations. This thorough workflow ensures both strong performance and robust understanding of model behavior.

This document is available on studocu

Downloaded by J Naresh Kumar (naresh5021@gmail.com)

7. Main computational challenges in large-scale deep learning and how they are addressed.

Large-scale deep learning faces challenges such as enormous model sizes, high memory consumption, long training times, and the need for fast data ingestion. GPUs and TPUs accelerate matrix multiplications and convolutions using highly parallel hardware. Distributed training strategies—data parallelism, model parallelism, pipeline parallelism, and parameter-server architectures—enable scaling across multiple devices or clusters. Mixed-precision training reduces memory usage and speeds up computation without sacrificing accuracy. Efficient input pipelines using caching, prefetching, and parallel workers alleviate I/O bottlenecks. Checkpointing, gradient accumulation, and careful scheduling ensure stability in massive models. Together, these techniques make it feasible to train billion-parameter networks on terabyte-scale datasets.

---

8. Major applications of deep learning in computer vision.

Deep learning has transformed computer vision across classification, detection, and segmentation. Image classification uses CNN architectures such as ResNet, EfficientNet, DenseNet, and Vision Transformers to assign semantic labels. Object detection relies on specialized models like Faster R-CNN, YOLO, SSD, and DETR that localize objects using bounding boxes and measure accuracy via mAP. Semantic and instance segmentation utilize architectures such as U-Net, DeepLab, Mask R-CNN, and Segmenter to produce fine-grained pixel-level predictions. Additional applications include image captioning, face recognition, super-resolution, and style transfer, each driven by specialized encoder–decoder or generative architectures. Deep learning's ability to learn hierarchical representations drives all these modern computer-vision successes.

---

9. Deep learning in speech recognition and generation.

Speech recognition uses representations like MFCCs, log-mel spectrograms, or raw-waveform features, combined with RNNs, CNNs, Transformers, or hybrid CTC-attention architectures to map audio to text. End-to-end systems such as DeepSpeech or wav2vec 2.0 learn directly from raw audio and align sequences using CTC or attention. Speech generation tasks—including TTS (text-to-speech) and voice conversion—use models like Tacotron, WaveNet, HiFi-GAN, FastSpeech, or diffusion-based vocoders to synthesize natural audio. These systems capture prosody, long-range dependencies, and phoneme-to-waveform mapping. Attention mechanisms and self-supervised pretraining have dramatically improved both recognition accuracy and generative quality, enabling near-human performance across many speech tasks.

---

10. Deep learning applications in NLP (translation, QA, language modeling).

Natural Language Processing has been revolutionized by architectures such as RNNs, Seq2Seq models, and more recently Transformers. Machine translation historically used encoder–decoder LSTM networks with attention, before being overtaken by Transformer-based systems such as BERT, GPT, and T5. Question answering relies on contextual representation models (BERT,

Downloaded by J Naresh Kumar (naresh5021@gmail.com)

RoBERTa) that encode semantic understanding of queries and passages. Language modeling is dominated by autoregressive transformers trained on massive corpora to predict the next token. NLP applications exploit self-attention to capture long-range dependencies, syntactic structure, and semantic relations, enabling state-of-the-art results across summarization, dialogue, paraphrasing, and information retrieval.

---

11. Other real-world applications of deep learning (recommenders, healthcare, robotics, finance).

Deep learning powers recommender systems (e.g., YouTube or Amazon) by learning user–item embeddings and nonlinear preference patterns from massive interaction logs. In healthcare, CNNs and Transformers interpret medical images, analyze EHR data, and support early diagnosis in radiology, cardiology, or genomics. Robotics employs reinforcement learning and vision-based policies to enable autonomous manipulation, navigation, and human–robot interaction. In finance, deep learning enables fraud detection, high-frequency trading prediction, portfolio optimization, and anomaly detection using temporal and multimodal data. These fields benefit from deep learning's ability to extract hierarchical patterns from noisy, high-dimensional datasets, providing strong generalization and automation capabilities.

This document is available on studocu

Downloaded by J Naresh Kumar (naresh5021@gmail.com)